FACULTAD DE INGENIERÍA

# REPORTE
## PARCIAL 2

# Quiz Time – Trivia

Interfaces Gráficas con Aplicaciones

M.I. NUÑEZ VARELA ALBERTO SALVADOR


Jimena Anais Sánchez Espino

Diego Emilio Ibarra Villela

Ana Laura Martínez Nava


Martes 24 de marzo 2026

## 1. Distribución de Tareas

| Integrante | Módulo | Actividades principales |
|---|---|---|
| Anais | Diseño de Interfaz | Diseño y construcción de pantallas (menú, categoría, juego). Organización de archivos y carpetas del proyecto. Integración de assets visuales. |
| Diego | Diseño de Interfaz / Lógica | Conexión de la base de datos a la interfaz. Implementación de lógica del juego: preguntas, aciertos/errores, modo aleatorio, pantalla de resultados. |
| Ana Laura | Base de Datos | Diseño y creación de la base de datos MySQL (tablas categorías y preguntas). Elaboración del reporte parcial. No pudo clonar el repositorio en Visual Studio. |

## 2. Descripción de Actividades

### Anais: Diseño de Interfaz

Se encargó del diseño visual de la aplicación. Diseñó y programó las pantallas principales del juego: pantalla de menú, pantalla de selección de categoría y pantalla de juego. Realizó la organización y ordenamiento de archivos y carpetas dentro del repositorio. También subió los assets e imágenes necesarias para la interfaz y realizó ajustes como la eliminación de botones innecesarios y cambios en la pantalla de categoría.

### Diego: Diseño de Interfaz y lógica del juego

Implementó la lógica funcional del juego en C#. Conectó la base de datos MySQL al formulario principal del juego, implementó el sistema de preguntas sin repetición y el contador de aciertos y errores. Agregó soporte para preguntas con imágenes, una animación de respuesta con un temporizador de 2 segundos para transicionar entre preguntas, la pantalla de resultados al finalizar el juego y el modo aleatorio que selecciona 10 preguntas al azar de cualquier categoría.

### Ana Laura – Base de Datos

Diseñó y creó la base de datos MySQL del proyecto. La base de datos contiene las tablas necesarias para almacenar las categorías y las preguntas de la trivia, incluyendo preguntas de opción múltiple y preguntas con imágenes. También se encargó de la elaboración del presente reporte. Debido a un problema técnico, no logró clonar el repositorio en Visual Studio, por lo que su trabajo de base de datos se integró al proyecto a través de pull requests gestionados por Anais.

## 3. Tecnologías Utilizadas

| Categoría | Tecnología | Descripción |
|---|---|---|
| IDE | Visual Studio | Entorno de desarrollo principal utilizado para programar la aplicación en C#. |
| Lenguaje | C# | Lenguaje de programación y framework utilizados para desarrollar el juego. |
| Manejador de BD | MySQL | Sistema gestor de base de datos relacional usado para almacenar categorías y preguntas de la trivia. |
| Biblioteca GUI | Canva (diseño) | Herramienta de diseño gráfico utilizada para crear los assets visuales e imágenes de la interfaz. |
| Control de versiones | Git / GitHub | Plataforma de control de versiones usada para gestionar el código fuente y la colaboración del equipo. |

## 4. Modelo Relacional de la Base de Datos

La base de datos se llama juego_trivia y está compuesta por dos tablas principales:

| Tabla | Campo | Tipo | Descripción |
|---|---|---|---|
| categorias | id | INT (PK, AI) | Identificador único de la categoría. |
| | nombre | VARCHAR(100) | Nombre de la categoría (ej. Música, Deportes). |
| preguntas | id | INT (PK, AI) | Identificador único de la pregunta. |
| | categoria_id | INT (FK) | Referencia a la tabla categorias. |
| | tipo | ENUM | Tipo de pregunta: opcion_multiple o imagen. |
| | pregunta | TEXT | Texto de la pregunta. |
| | url_pregunta_imagen | VARCHAR(500) | URL opcional si la pregunta incluye imagen de referencia. |
| | opcion_a / b / c / d | VARCHAR(500) | Texto o ruta de imagen para cada opción de respuesta. |
| | respuesta_correcta | CHAR(1) | Letra de la opción correcta (a, b, c o d). |

Relación: La tabla preguntas tiene una llave foránea (categoria_id) que referencia al campo id de la tabla categorias, con eliminación en cascada. Las categorías registradas son: Música, Deportes, Geografía y Viajes, Cine y Series, y Videojuegos y Tecnología.

## 5. Listado de Commits por Persona

### 5.1 Usuarios de GitHub

| Usuario de GitHub | Integrante |
|---|---|
| AnaisSanchez37 / Anais | Anais |
| Rendo / diegonavajas1-coder | Diego |
| alanaura229-jpg | Ana Laura |

### 5.2 Historial de Commits

| Usuario GitHub | Integrante | Fecha | Descripción |
|---|---|---|---|
| Rendo | Diego | 23/03/2026 | Regresar a la pantalla de categoría. |
| Rendo | Diego | 23/03/2026 | Se agregó la pantalla "juegoTerminado" con el resumen de la puntuación. |
| diegonavajas1-coder | Diego | 23/03/2026 | Animación de respuesta, se agregó un Timer de 2 segundos para transicionar entre preguntas. |
| Rendo | Diego | 20/03/2026 | Implementación del Modo Aleatorio. Se agregó el botón "Aleatorio" en la selección de categorías y se modificó la consulta SQL para traer 10 preguntas al azar. |
| diegonavajas1-coder | Diego | 16/03/2026 | Implementar preguntas con imágenes. |
| diegonavajas1-coder | Diego | 16/03/2026 | Se conectó la base de datos a FormJuego, se implementó la lista de preguntas sin repetir y el contador de aciertos/errores. |
| Anais | Anais | 16/03/2026 | Pantalla categoría agregada. |
| Anais | Anais | 16/03/2026 | Cambios en pantalla categoría. |
| Anais | Anais | 16/03/2026 | Eliminación de botón en pantalla menú. |
| Anais | Anais | 15/03/2026 | prueba |
| Anais | Anais | 15/03/2026 | Archivos ordenados. |
| Anais | Anais | 15/03/2026 | se ordenan carpetas. |
| Anais | Anais | 15/03/2026 | Base de Datos. |
| AnaisSanchez37 | Anais | 15/03/2026 | Merge branch 'main' into Base-de-Datos. |
| Anais | Anais | 15/03/2026 | Actualización. |
| Anais | Anais | 15/03/2026 | Archivos ordenados. |
| AnaisSanchez37 | Anais | 15/03/2026 | Merge pull request #2 from alanaura229-jpg/Base-de-Datos. |
| Anais | Anais | 15/03/2026 | se ordenan carpetas. |

| Usuario GitHub | Integrante | Fecha | Descripción |
|---|---|---|---|
| Anais | Anais | 15/03/2026 | Subiendo pantallas. |
| AnaisSanchez37 | Anais | 15/03/2026 | Merge pull request #1 from alanaura229-jpg/Base-de-Datos. |
| Anais | Anais | 15/03/2026 | Base de Datos. |
| alanaura229-jpg | Ana Laura | 13/03/2026 | base de datos creada. |

## 6. Conflictos Encontrados con Git/GitHub

### 6.1 Problema de clonación de repositorio

Ana Laura no pudo clonar el repositorio en Visual Studio, lo que le impidió trabajar directamente en el código del proyecto. Como solución, su trabajo de base de datos fue integrado al repositorio a través de pull requests (PR #1 y PR #2) gestionados desde la rama Base-de-Datos y mergeados por Anais Sánchez.

### 6.2 Gestión de ramas y merges

El proyecto utilizó múltiples ramas (main, Conexion, Base-de-Datos, Pantallas) para organizar el trabajo del equipo. Los merges entre ramas se realizaron mediante pull requests verificados en GitHub, lo que permitió integrar los cambios de forma ordenada a pesar del conflicto de clonación.

UASLP
Universidad Autónoma
de San Luis Potosí

**FACULTAD DE INGENIERÍA**

# REPORTE

## PARCIAL 3

## Quiz Time – Trivia

Interfaces Gráficas con Aplicaciones

M.I. NUÑEZ VARELA ALBERTO SALVADOR


Jimena Anais Sánchez Espino

Diego Emilio Ibarra Villela

Ana Laura Martínez Nava


Mayo 2026

## 1. Introducción

El presente reporte describe el desarrollo del sistema de trivia "Quiz Time – 100 Mexicanos Dijeron", una aplicación distribuida basada en arquitectura cliente-servidor que permite la participación de múltiples jugadores en tiempo real.

El sistema fue desarrollado utilizando C# con .NET WinForms para la interfaz gráfica, TCP Sockets para la comunicación en red y MySQL como sistema de almacenamiento de datos. Adicionalmente, se integró una API auxiliar en Python con Flask para la gestión dinámica de preguntas e imágenes.

Durante este parcial, el proyecto evolucionó de una aplicación local a un sistema multijugador completo, incorporando sincronización en red, manejo de concurrencia, persistencia de datos y soporte multimedia.

## 2. Objetivo del Proyecto

Desarrollar un sistema de trivia multijugador que permita:

- Conexión simultánea de múltiples jugadores.
- Sincronización en tiempo real de preguntas y respuestas.
- Gestión dinámica de preguntas desde base de datos.
- Registro de resultados y estadísticas por jugador.
- Interfaz gráfica interactiva y personalizada.

## 3. Distribución de Tareas

| Integrante | Módulo | Actividades principales |
|---|---|---|
| Anais | Diseño de Interfaz | Diseño de nuevas pantallas (FormConexion: pantalla de unirse a partida en red). Ajustes visuales en pantallas existentes. Organización del repositorio y gestión de pull requests. Elaboración del reporte parcial. |
| Diego | Servidor TCP / Lógica de Red | Desarrollo completo del servidor TCP (ServidorTrivia) en C#: manejo de múltiples clientes, sincronización del juego en red, broadcast de preguntas y resultados, guardado de estadísticas y partidas en MySQL. Implementación del cliente de red (FormJuegoRed) y API auxiliar en Python/Flask. |
| Ana Laura | Base de Datos / Lógica | Extensión del esquema MySQL: creación de tablas resultados_usuarios, detalle_respuestas y pregunta_estadisticas. Implementación de la selección de categoría por parte del host: el servidor envía la lista de categorías disponibles y el primer jugador (host) elige una antes de iniciar la partida. Elaboración del reporte parcial. |

# 4. Descripción de Actividades

## Anais – Diseño de Interfaz

Para el tercer parcial, Anais diseñó y programó la pantalla FormConexion, que permite al jugador ingresar su nombre y la IP del servidor para unirse a una partida en red. También realizó ajustes visuales en las pantallas existentes y se encargó de la organización del repositorio, gestionando los pull requests del equipo.

## Diego – Servidor TCP y Lógica de Red

Diego fue responsable de la implementación de la arquitectura de red del proyecto. Sus contribuciones principales fueron:

- Servidor TCP (ServidorTrivia, C# consola): escucha en el puerto 54321, acepta conexiones de clientes, maneja cada jugador en un hilo independiente y coordina el flujo completo del juego de forma sincronizada.
- Protocolo de mensajes JSON: todos los mensajes entre cliente y servidor usan System.Text.Json. Los tipos de mensaje implementados son: unirse, bienvenida, jugador_unido, inicio_juego, pregunta, respuesta, resultado_pregunta y fin_juego.
- Lógica de juego en red: carga 10 preguntas aleatorias de la BD al iniciar, envía cada pregunta a todos los clientes, espera respuestas (máximo 30 segundos por pregunta), evalúa resultados y envía retroalimentación personalizada a cada jugador.
- Guardado de resultados: al finalizar la partida, el servidor guarda el registro en resultados_usuarios y actualiza pregunta_estadisticas por cada pregunta respondida usando MySql.Data.
- FormJuegoRed (cliente WinForms): pantalla de juego adaptada para recibir preguntas por red y enviar respuestas al servidor.
- API auxiliar Python/Flask (TriviaAPI/api.py): expone el endpoint GET /api/preguntas (retorna 10 preguntas aleatorias en JSON) y el endpoint GET /imagenes/<ruta> para servir imágenes desde el servidor local (puerto 8000).

## Ana Laura – Base de Datos y Selección de Categoría por Host

Ana Laura extendió el script trivia.sql con las tres nuevas tablas requeridas para el modo de red: resultados_usuarios, detalle_respuestas y pregunta_estadisticas. Además implementó la funcionalidad de selección de categoría por parte del host: al conectarse el primer jugador, el servidor le envía la lista de categorías disponibles (consultadas desde la BD) para que elija una antes de que inicie la partida. El servidor filtra las preguntas según la categoría seleccionada por el host y comunica la decisión al resto de los jugadores.

# 5. Arquitectura de la Solución en Red

## 5.1 Servidor TCP (ServidorTrivia)

El servidor es una aplicación de consola C# que corre de forma independiente al cliente WinForms. Sus responsabilidades son:

- Escuchar conexiones entrantes en el puerto 54321 con TcpListener.
- Crear un hilo (Thread) por cada cliente conectado para manejarlos de forma concurrente.
- Gestionar el lobby: espera a que se conecte el número requerido de jugadores (MAX_JUGADORES) antes de iniciar la partida.
- Ejecutar el flujo del juego en un hilo separado (EjecutarJuego): envía preguntas, espera respuestas con timeout de 30 segundos, calcula resultados y hace broadcast a todos los clientes.

• Persistir resultados en MySQL: GuardarEstadisticaPregunta() inserta en pregunta_estadisticas y GuardarPartidaFinal() inserta en resultados_usuarios con el categoria_id de la categoría seleccionada por el host al concluir la partida.

• Mostrar en consola el estado del servidor con colores: conexiones, respuestas de jugadores, errores y resultado final.

## 5.2 Protocolo de Mensajes JSON

La comunicación usa mensajes de texto JSON enviados por TCP. Los tipos de mensaje y su dirección son:

| Tipo de mensaje | Dirección | Descripción |
|---|---|---|
| **unirse** | Cliente → Servidor | El jugador envía su nombre para registrarse en el lobby. |
| **bienvenida** | Servidor → Cliente | Confirma la unión: asigna ID, lista de jugadores actuales y cuántos faltan. |
| **jugador_unido** | Servidor → Resto | Notifica a los demás cuando se une un nuevo jugador. |
| **error** | Servidor → Cliente | Rechaza la conexión si el juego ya inició o está lleno. |
| **lista_categorias** | Servidor → Host | Envía al host la lista de categorías disponibles para que elija una. |
| **seleccionar_categoria** | Host → Servidor | El host envía el id de la categoría elegida. |
| **categoria_seleccionada** | Servidor → Todos | Notifica a todos los jugadores la categoría que se jugará. |
| **inicio_juego** | Servidor → Todos | Señal de que la partida comienza. |
| **pregunta** | Servidor → Todos | Envía número, texto, opciones y tipo de la pregunta actual. |
| **respuesta** | Cliente → Servidor | El jugador envía el inciso seleccionado (a, b, c o d). |
| **resultado_pregunta** | Servidor → Cada jugador | Indica la respuesta correcta, si el jugador acertó y las respuestas de todos. |
| **fin_juego** | Servidor → Todos | Envía estadísticas finales (aciertos/errores) y nombre del ganador. |

## 5.3 API Auxiliar Python/Flask (TriviaAPI)

Además del servidor TCP, se implementó una API HTTP con Flask que corre en el puerto 8000 de la misma máquina que el servidor. Expone dos endpoints:

• GET /api/preguntas: consulta la BD y retorna 10 preguntas aleatorias en formato JSON (id, tipo, pregunta, opciones, respuesta_correcta).

• GET /imagenes/<ruta>: sirve imágenes desde la carpeta local C:\Imag, permitiendo que los clientes remotos carguen las imágenes de las preguntas tipo imagen via HTTP.

• POST /api/estadisticas: Recibe un objeto JSON con el ID de la pregunta y el conteo de respuestas correctas e incorrectas de los jugadores para generar analíticas de dificultad.

• POST /api/resultados: Registra el desenlace de la partida en la base de datos, incluyendo los nombres de los participantes, sus puntajes finales y el ganador, permitiendo persistir el historial de juegos.

## 6. Modelo Relacional de la Base de Datos

La base de datos juego_trivia fue extendida en el tercer parcial para soportar el modo multijugador en red. Ahora comprende cinco tablas: categorias, preguntas, más las nuevas resultados_usuarios, detalle_respuestas y pregunta_estadisticas.

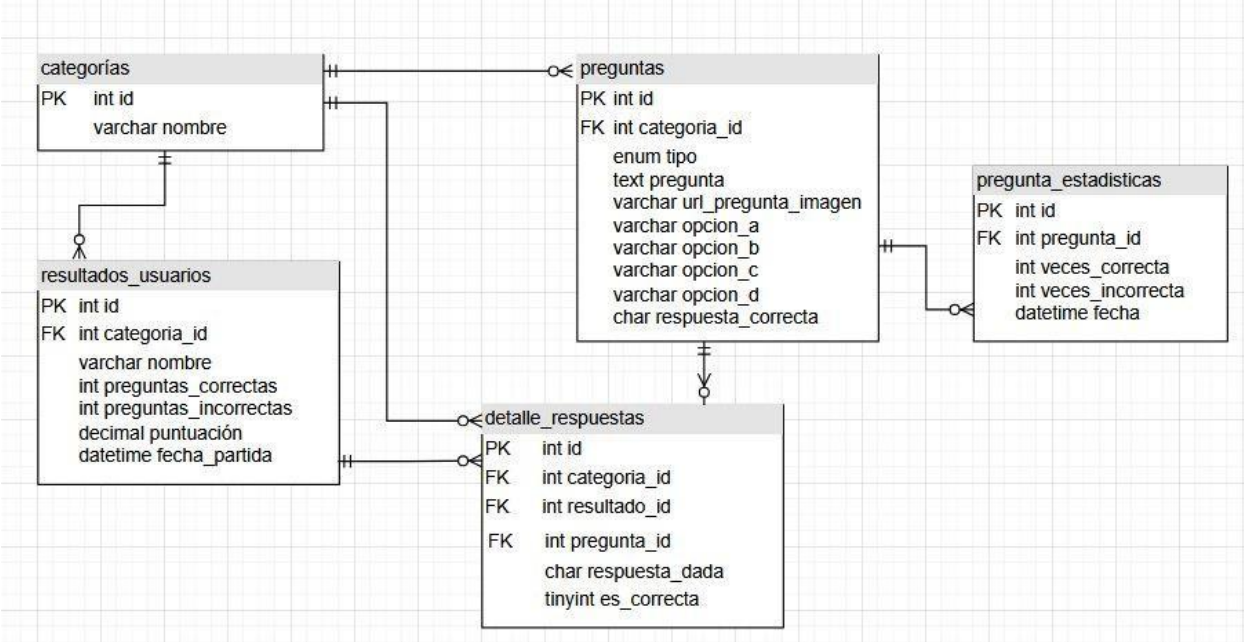
| Tabla | Campo | Tipo | Descripción |
|---|---|---|---|
| **categorias** | id | INT PK AI | Identificador único de la categoría. |
| | nombre | VARCHAR(100) | Nombre de la categoría (Música, Deportes, etc.). |
| **preguntas** | id | INT PK AI | Identificador único de la pregunta. |
| | categoria_id | INT FK | Referencia a categorias(id) ON DELETE CASCADE. |
| | tipo | ENUM | opcion_multiple o imagen. |
| | pregunta | TEXT | Texto de la pregunta. |
| | url_pregunta_imagen | VARCHAR(500) | URL de imagen si la pregunta la requiere. |
| | opcion_a/b/c/d | VARCHAR(500) | Texto o ruta de imagen para cada opción. |
| | respuesta_correcta | CHAR(1) | Letra correcta: a, b, c o d. |
| **resultados_usuarios** | id | INT PK AI | Identificador único de la partida guardada. |
| | nombre | VARCHAR(100) | Nombre del jugador. |
| | categoria_id | INT FK NULL | Categoría jugada (NULL en modo red/aleatorio). |
| | preguntas_correctas | INT | Número de respuestas correctas. |
| | preguntas_incorrectas | INT | Número de respuestas incorrectas. |
| | puntuacion | DECIMAL(5,2) | Puntuación final (aciertos × 100). |
| | fecha_partida | DATETIME | Fecha y hora de la partida. |
| **detalle_respuestas** | id | INT PK AI | Identificador único del registro. |
| | resultado_id | INT FK | Referencia a resultados_usuarios(id). |
| | pregunta_id | INT FK | Referencia a preguntas(id). |
| | categoria_id | INT FK | Categoría de la pregunta respondida. |
| | respuesta_dada | CHAR(1) | Opción seleccionada por el jugador. |
| | es_correcta | TINYINT(1) | 1 = correcta, 0 = incorrecta. |
| **pregunta_estadisticas** | id | INT PK AI | Identificador del registro estadístico. |
| | pregunta_id | INT FK | Referencia a preguntas(id) ON DELETE CASCADE. |
| | veces_correcta | INT | Veces que fue respondida correctamente. |

| | | | |
|---|---|---|---|
| | veces_incorrecta | INT | Veces que fue respondida incorrectamente. |
| | fecha | DATETIME | Fecha del registro estadístico. |

Relaciones entre tablas:

- preguntas.categoria_id → categorias.id
- resultados_usuarios.categoria_id → categorias.id
- detalle_respuestas.resultado_id → resultados_usuarios.id
- detalle_respuestas.pregunta_id → preguntas.id
- detalle_respuestas.categoria_id → categorias.id
- pregunta_estadisticas.pregunta_id → preguntas.id

### 6.1 Diagrama Entidad-Relación




## 7. Tecnologías Utilizadas

| Categoría | Tecnología | Descripción |
|---|---|---|
| IDE | Visual Studio 2022 | Entorno de desarrollo principal para C# (.NET Framework 4.7.2). |
| Lenguaje | C# / WinForms | Lenguaje y framework para la aplicación de escritorio del cliente. |

| | | |
|---|---|---|
| Lenguaje | C# / Consola | Lenguaje del servidor TCP (proyecto ServidorTrivia, consola .NET). |
| Lenguaje | Python 3 / Flask | API auxiliar para servir preguntas e imágenes vía HTTP (puerto 8000). |
| Base de Datos | MySQL | Almacena categorías, preguntas, resultados_usuarios, detalle_respuestas y pregunta_estadisticas. |
| Protocolo de Red | TCP Sockets (puerto 54321) | Comunicación cliente-servidor en tiempo real usando TcpListener / TcpClient. |
| Formato de Mensajes | JSON (System.Text.Json) | Serialización de todos los mensajes entre cliente y servidor. |
| Diseño gráfico | Canva | Creación de assets visuales e imágenes de la interfaz. |
| Control de versiones | Git / GitHub | Gestión de código con ramas main, pantallas, Conexion y conexion-final. |

# 8. Historial de Commits

## 8.1 Usuarios de GitHub

| Usuario de GitHub | Integrante |
|---|---|
| AnaisSanchez37 / Anais | Anais |
| Rendo / diegonavajas1-coder | Diego |
| alanaura229-jpg | Ana Laura |

## 8.2 Historial de Commits (P3 + P2)

| Usuario GitHub | Integrante | Fecha | Descripción |
|---|---|---|---|
| DiegoVillela17 | Diego | 02/05/2026 | Se restauró la interfaz de selección de categorías en el lobby (Rama: IntentoFinalPruebas). |
| DiegoVillela17 | Diego | 02/05/2026 | Se quitó SQL de C#, ahora todo el juego usa la API de Python y organización de archivos. |
| alanaura229-jpg | Ana Laura | 01/05/2026 | Base de datos corregida, nuevas tablas agregadas (Rama: Base-de-Datos). |
| DiegoVillela17 | Diego | 01/05/2026 | Refactorización del lobby: Limpieza de código y conexión de IP dinámica. |

| | | | |
|---|---|---|---|
| DiegoVillela17 | Diego | 01/05/2026 | Cambio Multijugador: Detección de Empate y transición de API local a Sockets. |
| DiegoVillela17 | Diego | 30/04/2026 | Organización parte 2: SQL y API (Rama: IntentoFinalPruebas). |
| DiegoVillela17 | Diego | 30/04/2026 | Commit inicial rama IntentoFinalPruebas: Limpieza de estructura. |
| DiegoVillela17 | Diego | 30/04/2026 | Cambios en JuegoRed (Rama: conexion-final). |
| Rendo | Diego | 29/04/2026 | conexion-final: Servidor TCP con manejo de clientes, broadcast JSON, guardado en BD. |
| AnaisSanchez37 | Anais | 29/04/2026 | Intento para cargar imágenes / Modificación para cargar las imágenes. |
| alanaura229-jpg | Ana Laura | 29/04/2026 | Implementación de selección de categoría por el host: servidor envía lista de categorías y filtra preguntas según la elección. |
| diegonavajas1- coder | Diego | 28/04/2026 | Implementación de API Python/Flask (TriviaAPI/api.py) y actualización de rutas SQL relativas. |
| diegonavajas1- coder | Diego | 28/04/2026 | Pantallas de red: FormConexion (unirse a partida) y FormJuegoRed (juego en red). |
| alanaura229-jpg | Ana Laura | 28/04/2026 | Extensión de trivia.sql: tablas resultados_usuarios, detalle_respuestas, pregunta_estadisticas. |
| AnaisSanchez37 | Anais | 27/04/2026 | Lógica del servidor y cliente (Rama: pantallas). |
| Rendo | Diego | 23/03/2026 | Regresar a la pantalla de categoría. |
| Rendo | Diego | 23/03/2026 | Se agregó la pantalla juegoTerminado con el resumen de la puntuación. |

| | | | |
|---|---|---|---|
| diegonavajas1- coder | Diego | 23/03/2026 | Animación de respuesta: Timer de 2 seg para transicionar entre preguntas. |
| AnaisSanchez37 | Anais | 23/03/2026 | Cambios en el diseño de pantalla historial y agregación de código. |
| AnaisSanchez37 | Diego | 20/03/2026 | Implementación del Modo Aleatorio: botón en selección de categorías, consulta SQL aleatoria. |
| diegonavajas1- coder | Diego | 16/03/2026 | Implementar preguntas con imágenes. |
| diegonavajas1- coder | Diego | 16/03/2026 | Conexión BD a FormJuego, lista de preguntas sin repetir y contador de aciertos/errores. |
| AnaisSanchez37 | Anais | 16/03/2026 | Pantalla categoría agregada y cambios en pantalla categoría. |
| AnaisSanchez37 | Anais | 15/03/2026 | Archivos ordenados y carpetas organizadas. |
| alanaura229-jpg | Ana Laura | 13/03/2026 | base de datos creada (tablas categorias y preguntas). |

## 8.3 Gráfica de ramas creadas

## 8.4 Todos los commits

Se restauró la interfaz de selección de categorías en el lobby
DiegoVillela17 pushed 1 commit to IntentoFinalPruebas • cb2c551...01326e8 • 3 hours ago

Se quitó SQL de C#, ahora todo el juego usa la API de Python, Organiz... Force push
DiegoVillela17 force pushed to IntentoFinalPruebas • ebb1dcc...cb2c551 • 4 hours ago

base de datos corregida, nuevas tablas agregadas
alanaura229-jpg pushed 1 commit to Base-de-Datos • e98304e...f38bb3e • 22 hours ago

Refactorización del lobby: Limpieza de código y conexión de IP dinámi...
DiegoVillela17 pushed 1 commit to IntentoFinalPruebas • 3f594ac...ebb1dcc • yesterday

Cambio Multijugador Detección de Empate, Transición API LOCAL a SOCKE...
DiegoVillela17 pushed 1 commit to IntentoFinalPruebas • 27a5e77...3f594ac • yesterday

Organización parte 2: SQL y API
DiegoVillela17 pushed 1 commit to IntentoFinalPruebas • 5137355...27a5e77 • 2 days ago

Commit inicial rama IntentoFinalPruebas: Limpieza de estructura y eli...
DiegoVillela17 created IntentoFinalPruebas • 5137355 • 2 days ago

Cambios en JuegoRed
AnaisSanchez37 created conexion-final • cd69d13 • 2 days ago

cambios de API
alanaura229-jpg pushed 1 commit to main • f0d2802...a2b4da8 • 3 days ago

Intento para cargar imegenes
AnaisSanchez37 pushed 1 commit to pantallas • 873b822...d4150de • 3 days ago

Modificación para cargar las imagenes
AnaisSanchez37 pushed 1 commit to pantallas • b6f6259...873b822 • 3 days ago

Merge branch 'pantallas' of https://github.com/alanaura229-jpg/100mex...
AnaisSanchez37 pushed 2 commits to pantallas • 98fa4eb...b6f6259 • 3 days ago

Implementacion de API Python (Prueba) y cambios en SQL con rutas rela...
DiegoVillela17 pushed 1 commit to Conexion • 832d654...168c126 • 4 days ago

Probando
DiegoVillela17 pushed 1 commit to pantallas • b3ef5d6...98fa4eb • 4 days ago

Cambios para cargar preguntas
AnaisSanchez37 pushed 1 commit to pantallas • 26c2466...b3ef5d6 • 5 days ago

lógica del servidory cliente
AnaisSanchez37 pushed 1 commit to pantallas • fb17ece...26c2466 • 6 days ago

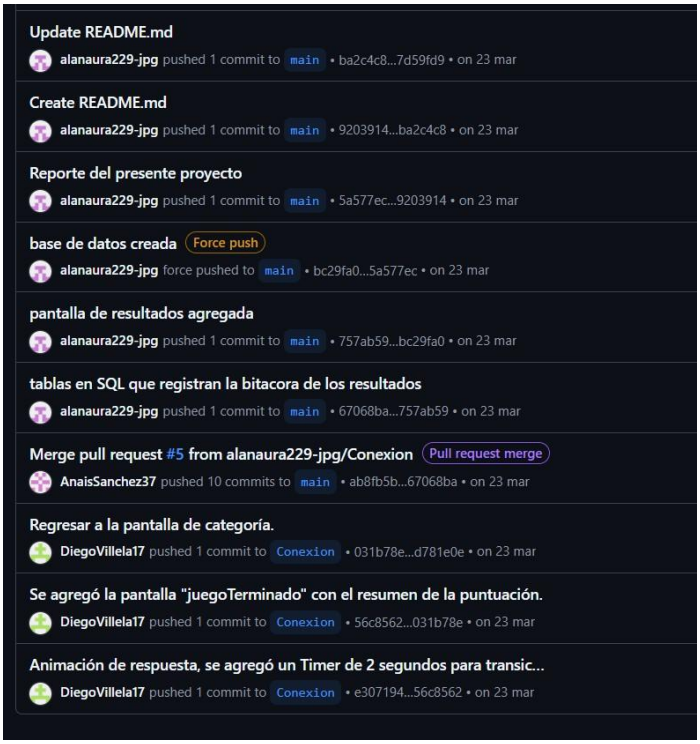
historial del juego
alanaura229-jpg pushed 1 commit to Conexion • d781e0e...832d654 • on 24 mar

Merge pull request #6 from alanaura229-jpg/pantallas Pull request merge
AnaisSanchez37 pushed 27 commits to main • 7d59fd9...f0d2802 • on 24 mar

Cambios en el Diseño de pantalla historial
AnaisSanchez37 pushed 1 commit to pantallas • 9e6c749...fb17ece • on 23 mar

Agrgación de código para el historial
AnaisSanchez37 created pantallas • 9e6c749 • on 23 mar

Update README.md
alanaura229-jpg pushed 1 commit to main • ba2c4c8...7d59fd9 • on 23 mar




### 8.5 Link del repositorio

https://github.com/alanaura229-jpg/100mexicanosDijeron.git

## 9. Conflictos y Soluciones

### 9.1 Problema de clonación de repositorio

Ana Laura continuó sin poder clonar el repositorio directamente en Visual Studio. Su trabajo de base de datos fue integrado mediante pull requests gestionados por Anais desde la rama Base-de-Datos.

### 9.2 Gestión de ramas

Para el tercer parcial se utilizaron las ramas pantallas (trabajo de Diego en prueba inicial con ServidorTrivia), Conexion (integración de API Python y ajustes SQL) y conexion-final (branch de trabajo final del cliente de red). Los merges se realizaron mediante pull requests en GitHub.

### 9.3 Credenciales en código

Las cadenas de conexión a MySQL (usuario root, contraseña en texto plano) y la ruta fija de imágenes (C:\Imag) están hardcodeadas tanto en el servidor C# como en la API Python. Para un ambiente de producción sería necesario usar variables de entorno o un archivo de configuración externo.

### 9.4 Sincronización de respuestas

Se implementaron mecanismos de control (locks) en el servidor para evitar condiciones de carrera cuando varios jugadores responden simultáneamente, garantizando la correcta evaluación de respuestas.

### 9.5 Manejo de rutas de imágenes

Se convirtieron las rutas locales a formato URL, permitiendo que los clientes remotos accedan a las imágenes vía HTTP a través de la API Flask, eliminando la dependencia de rutas locales.

## 10. Conclusión

El proyecto "Quiz Time – 100 Mexicanos Dijeron" logró consolidarse como un sistema distribuido funcional que integra múltiples tecnologías modernas. Se consiguió comunicación en tiempo real entre clientes y servidor, manejo concurrente de múltiples jugadores, persistencia eficiente de datos y una interfaz gráfica interactiva.

El trabajo en equipo con roles claramente definidos —Anais en interfaz, Diego en arquitectura de red y Ana Laura en base de datos— y el uso de control de versiones con GitHub permitieron una evolución organizada y paralela del sistema.

Este proyecto representa un avance importante en el aprendizaje de conceptos como programación en red, concurrencia, bases de datos relacionales e interfaces gráficas, sentando bases sólidas para futuros desarrollos más complejos.